Partie 1 - Code de l'interface (app.py)

```
import tkinter as tk
from tkinter import ttk, messagebox

import ciphers
import integrity
import passwords


def read_hill_key(text):
    if "," in text:
        numbers = [int(x) for x in text.split(",")]
        size = int(len(numbers) ** 0.5)
        if size * size != len(numbers):
            raise ValueError("la matrice doit être carrée")
        return [numbers[i * size:(i + 1) * size] for i in range(size)]
    return ciphers.key_from_text(text)


def read_caesar_key(text):
    try:
        return int(text)
    except ValueError:
        raise ValueError("clé invalide : entrez un nombre entier")


def make_text_box(parent, height):
    box = tk.Text(parent, height=height, wrap="word")
    box.pack(fill="both", expand=True)
    return box


def set_text(box, text):
    box.config(state="normal")
    box.delete("1.0", "end")
    box.insert("1.0", text)
    box.config(state="disabled")


def get_text(box):
    return box.get("1.0", "end").strip()


class MessagingTab(ttk.Frame):
    def __init__(self, parent):
        super().__init__(parent, padding=10)

        settings = ttk.Frame(self)
        settings.pack(fill="x")
        ttk.Label(settings, text="Algorithme").grid(row=0, column=0)
        self.algorithm = ttk.Combobox(settings, values=["Caesar", "Hill", "Playfair"],
            state="readonly", width=12)
        self.algorithm.current(0)
        self.algorithm.grid(row=0, column=1, padx=6)
        self.algorithm.bind("<<ComboboxSelected>>", self.change_algorithm)
        ttk.Label(settings, text="Clé").grid(row=0, column=2)
        self.key = ttk.Entry(settings, width=22)
        self.key.grid(row=0, column=3, padx=6)
        ttk.Label(settings, text="Sens").grid(row=0, column=4)
        self.direction = ttk.Combobox(settings, values=["droite", "gauche"], state="readonly",
            width=8)
        self.direction.current(0)
        self.direction.grid(row=0, column=5, padx=6)

        self.hint = ttk.Label(self, foreground="#555")
        self.hint.pack(anchor="w", pady=6)

        secret_row = ttk.Frame(self)
        secret_row.pack(fill="x")
        ttk.Label(secret_row, text="Secret partagé (HMAC)").pack(side="left")
        self.secret = ttk.Entry(secret_row, width=26, show="*")
        self.secret.insert(0, "usthb2026")
```

Partie 1 - Code de l'interface (app.py)

```
        self.secret.pack(side="left", padx=6)

        frame_a = ttk.LabelFrame(self, text="Entité A", padding=6)
        frame_a.pack(fill="both", expand=True, pady=6)
        self.message = make_text_box(frame_a, 4)
        self.message.insert("1.0", "Retrouve-moi a la bibliotheque a midi")
        ttk.Button(frame_a, text="Chiffrer et envoyer", command=self.send).pack(anchor="e", pady=4)

        channel = ttk.LabelFrame(self, text="Canal (modifiable pour simuler un attaquant)",
            padding=6)
        channel.pack(fill="both", expand=True, pady=6)
        self.cipher_box = make_text_box(channel, 3)
        tag_row = ttk.Frame(channel)
        tag_row.pack(fill="x", pady=4)
        ttk.Label(tag_row, text="Tag").pack(side="left")
        self.tag_entry = ttk.Entry(tag_row)
        self.tag_entry.pack(side="left", fill="x", expand=True, padx=6)
        ttk.Button(tag_row, text="Changer une lettre", command=self.tamper).pack(side="left")

        frame_b = ttk.LabelFrame(self, text="Entité B", padding=6)
        frame_b.pack(fill="both", expand=True, pady=6)
        self.result_box = make_text_box(frame_b, 4)
        self.result_box.config(state="disabled")
        bottom = ttk.Frame(frame_b)
        bottom.pack(fill="x", pady=4)
        self.status = ttk.Label(bottom)
        self.status.pack(side="left")
        ttk.Button(bottom, text="Vérifier et déchiffrer", command=self.receive).pack(side="right")

        self.change_algorithm()

    def clear_channel(self):
        self.cipher_box.delete("1.0", "end")
        self.tag_entry.delete(0, "end")
        set_text(self.result_box, "")
        self.status.config(text="")

    def change_algorithm(self, event=None):
        self.clear_channel()
        name = self.algorithm.get()
        self.key.delete(0, "end")
        if name == "Caesar":
            self.key.insert(0, "3")
            self.hint.config(text="Clé : décalage n (entier). Le sens s'applique au chiffrement.")
            self.direction.config(state="readonly")
        elif name == "Hill":
            self.key.insert(0, "3,3,2,5")
            self.hint.config(text="Clé : 4 ou 9 nombres séparés par des virgules, ou un mot de 4 ou
                9 lettres.")
            self.direction.config(state="disabled")
        else:
            self.key.insert(0, "monarchy")
            self.hint.config(text="Clé : un mot quelconque (J est confondu avec I).")
            self.direction.config(state="disabled")

    def encrypt(self, text):
        name = self.algorithm.get()
        key = self.key.get()
        if name == "Caesar":
            shift = read_caesar_key(key)
            ciphers.check_caesar_key(shift)
            return ciphers.caesar(text, shift, self.direction.get())
        if name == "Hill":
            hill_key = read_hill_key(key)
            ciphers.check_hill_key(hill_key)
            return ciphers.hill_encrypt(text, hill_key)
        ciphers.check_playfair_key(key)
        return ciphers.playfair_encrypt(text, key)

    def decrypt(self, text):
        name = self.algorithm.get()
```

Partie 1 - Code de l'interface (app.py)

```
        key = self.key.get()
        if name == "Caesar":
            return ciphers.caesar_decrypt(text, read_caesar_key(key), self.direction.get())
        if name == "Hill":
            hill_key = read_hill_key(key)
            return ciphers.hill_strip_padding(ciphers.hill_decrypt(text, hill_key), len(hill_key))
        return ciphers.playfair_strip_padding(ciphers.playfair_decrypt(text, key))

    def send(self):
        try:
            encrypted = self.encrypt(get_text(self.message))
        except ValueError as error:
            self.clear_channel()
            self.status.config(text="rien n'a été envoyé : " + str(error), foreground="red")
            return
        self.clear_channel()
        self.cipher_box.insert("1.0", encrypted)
        self.tag_entry.insert(0, integrity.tag(encrypted, self.secret.get()))
        self.status.config(text="message envoyé", foreground="green")

    def tamper(self):
        text = get_text(self.cipher_box)
        if not text:
            return
        middle = len(text) // 2
        new_letter = "B" if text[middle] == "A" else "A"
        self.cipher_box.delete("1.0", "end")
        self.cipher_box.insert("1.0", text[:middle] + new_letter + text[middle + 1:])

    def receive(self):
        encrypted = get_text(self.cipher_box)
        received_tag = self.tag_entry.get().strip()
        if not encrypted:
            self.status.config(text="aucun message reçu : A doit d'abord envoyer", foreground="red")
            return
        if not integrity.verify(encrypted, self.secret.get(), received_tag):
            self.status.config(text="intégrité non vérifiée, message rejeté", foreground="red")
            set_text(self.result_box, "")
            return
        try:
            decrypted = self.decrypt(encrypted)
        except ValueError as error:
            messagebox.showerror("Erreur", str(error))
            return
        self.status.config(text="message authentique", foreground="green")
        set_text(self.result_box, decrypted)


class CipherAttackTab(ttk.Frame):
    def __init__(self, parent):
        super().__init__(parent, padding=10)

        ttk.Label(self, text="Texte chiffré").pack(anchor="w")
        self.cipher_box = tk.Text(self, height=5, wrap="word")
        self.cipher_box.pack(fill="x")
        self.cipher_box.insert("1.0", ciphers.caesar("Il faut garder le secret entre nous trois",
            11))

        buttons = ttk.Frame(self)
        buttons.pack(fill="x", pady=6)
        ttk.Button(buttons, text="César : les 26 décalages",
            command=self.caesar_all).pack(side="left")
        ttk.Button(buttons, text="César : analyse de fréquence",
            command=self.caesar_frequency).pack(side="left", padx=6)
        ttk.Button(buttons, text="Playfair : comptage des digrammes",
            command=self.digrams).pack(side="left")

        hill = ttk.LabelFrame(self, text="Hill 2x2 : attaque à clair connu", padding=6)
        hill.pack(fill="x", pady=6)
        ttk.Label(hill, text="Clair connu").grid(row=0, column=0, sticky="w")
        self.known_plain = ttk.Entry(hill, width=30)
```

Partie 1 - Code de l'interface (app.py)

```
        self.known_plain.insert(0, "MEETMEATNOON")
        self.known_plain.grid(row=0, column=1, padx=6)
        ttk.Label(hill, text="Chiffré correspondant").grid(row=1, column=0, sticky="w")
        self.known_cipher = ttk.Entry(hill, width=30)
        self.known_cipher.insert(0, ciphers.hill_encrypt("MEETMEATNOON", [[3, 3], [2, 5]]))
        self.known_cipher.grid(row=1, column=1, padx=6)
        ttk.Button(hill, text="Retrouver la clé", command=self.hill_attack).grid(row=0, column=2,
            rowspan=2, padx=6)

        self.output = tk.Text(self, height=14, wrap="word", state="disabled")
        self.output.pack(fill="both", expand=True)

    def caesar_all(self):
        lines = []
        for shift, text in ciphers.caesar_bruteforce(get_text(self.cipher_box)):
            lines.append("%2d  %s" % (shift, text))
        set_text(self.output, "\n".join(lines))

    def caesar_frequency(self):
        text = get_text(self.cipher_box)
        shift = ciphers.caesar_frequency_guess(text)
        clear = ciphers.caesar(text, shift, "left")
        set_text(self.output, "La lettre la plus fréquente suggère le décalage %d\n\n%s" % (shift,
            clear))

    def digrams(self):
        counts = ciphers.digram_counts(get_text(self.cipher_box))[:15]
        if not counts:
            set_text(self.output, "texte insuffisant")
            return
        set_text(self.output, "\n".join("%s  %d" % (pair, count) for pair, count in counts))

    def hill_attack(self):
        try:
            key = ciphers.hill_known_plaintext(self.known_plain.get(), self.known_cipher.get(), 2)
        except ValueError as error:
            set_text(self.output, str(error))
            return
        set_text(self.output, "Matrice clé retrouvée :\n%s\n%s" % (key[0], key[1]))


CASES = {
    "3 caractères, chacun 0 ou 1": (passwords.BINARY, 3),
    "4 caractères, chacun de 1 à 4": (passwords.DIGITS_1_4, 4),
    "caractères a-z A-Z 0-9 + * ...": (passwords.FULL, 3),
}


class PasswordTab(ttk.Frame):
    def __init__(self, parent):
        super().__init__(parent, padding=10)

        top = ttk.Frame(self)
        top.pack(fill="x")
        ttk.Label(top, text="Cas").grid(row=0, column=0, sticky="w")
        self.case = ttk.Combobox(top, values=list(CASES), state="readonly", width=32)
        self.case.current(0)
        self.case.grid(row=0, column=1, padx=6)
        self.case.bind("<<ComboboxSelected>>", self.change_case)
        ttk.Label(top, text="Mot de passe cible").grid(row=1, column=0, sticky="w", pady=6)
        self.target = ttk.Entry(top, width=24)
        self.target.grid(row=1, column=1, sticky="w", padx=6)

        buttons = ttk.Frame(self)
        buttons.pack(fill="x", pady=6)
        ttk.Button(buttons, text="Attaque par dictionnaire",
            command=self.run_dictionary).pack(side="left")
        ttk.Button(buttons, text="Force brute", command=self.run_brute_force).pack(side="left",
            padx=6)
        ttk.Button(buttons, text="Dictionnaire puis force brute",
            command=self.run_both).pack(side="left")
```

Partie 1 - Code de l'interface (app.py)

```
272
273         self.output = tk.Text(self, height=18, wrap="word", state="disabled")
274         self.output.pack(fill="both", expand=True)
275         self.change_case()
276
277     def change_case(self, event=None):
278         charset, length = CASES[self.case.get()]
279         self.target.delete(0, "end")
280         self.target.insert(0, charset[1] * length)
281
282     def read_target(self):
283         charset, length = CASES[self.case.get()]
284         password = self.target.get()
285         if len(password) != length or not set(password) <= set(charset):
286             messagebox.showwarning("Mot de passe", "Le mot de passe doit avoir %d caractères pris
                dans l'ensemble autorisé." % length)
287             return None
288         return password
289
290     def show_result(self, method, found, tries, seconds):
291         duration = passwords.human_time(seconds)
292         if found is None:
293             line = "%s : introuvable après %d essais (%s)" % (method, tries, duration)
294         else:
295             line = "%s : '%s' trouvé après %d essais (%s)" % (method, found, tries, duration)
296         self.output.config(state="normal")
297         self.output.insert("end", line + "\n")
298         self.output.config(state="disabled")
299
300     def run_dictionary(self):
301         password = self.read_target()
302         if password:
303             found, tries, seconds = passwords.dictionary_attack(passwords.sha(password))
304             self.show_result("dictionnaire", found, tries, seconds)
305
306     def run_brute_force(self):
307         password = self.read_target()
308         if password:
309             charset, length = CASES[self.case.get()]
310             found, tries, seconds = passwords.brute_force(passwords.sha(password), charset, length)
311             self.show_result("force brute", found, tries, seconds)
312
313     def run_both(self):
314         password = self.read_target()
315         if password:
316             charset, length = CASES[self.case.get()]
317             found, tries, seconds, method = passwords.combined_attack(passwords.sha(password),
                charset, length)
318             self.show_result(method, found, tries, seconds)
319
320
321 class ProtectionTab(ttk.Frame):
322     def __init__(self, parent):
323         super().__init__(parent, padding=10)
324
325         top = ttk.Frame(self)
326         top.pack(fill="x")
327         ttk.Label(top, text="Mot de passe").pack(side="left")
328         self.password = ttk.Entry(top, width=26)
329         self.password.insert(0, "Tr0ub4dor*93x")
330         self.password.pack(side="left", padx=6)
331         ttk.Button(top, text="Tester la robustesse", command=self.check_strength).pack(side="left")
332         ttk.Button(top, text="Mesurer le hachage lent",
                command=self.measure_hash).pack(side="left", padx=6)
333         ttk.Button(top, text="Démo du verrouillage", command=self.lockout_demo).pack(side="left")
334         ttk.Button(top, text="Tableau des temps d'attaque",
                command=self.time_table).pack(side="left", padx=6)
335
336         self.output = tk.Text(self, height=22, wrap="word", state="disabled")
337         self.output.pack(fill="both", expand=True, pady=8)
338
```

Partie 1 - Code de l'interface (app.py)

```
    def check_strength(self):
        problems = passwords.strength_problems(self.password.get())
        if problems:
            set_text(self.output, "Refusé :\n- " + "\n- ".join(problems))
        else:
            set_text(self.output, "Accepté.")

    def measure_hash(self):
        import time
        record = passwords.make_record(self.password.get())
        start = time.perf_counter()
        passwords.check_record(record, "wrong")
        slow_time = time.perf_counter() - start
        fast_rate = passwords.hash_rate(50000)
        slow_rate = 1 / slow_time
        text = "Une vérification PBKDF2 (%d tours) : %s\n" % (record["iterations"],
            passwords.human_time(slow_time))
        text += "SHA-256 simple : environ %d essais par seconde\n" % fast_rate
        text += "Hachage lent : environ %d essais par seconde\n" % slow_rate
        text += "Facteur de ralentissement : environ %dx" % (fast_rate / slow_rate)
        set_text(self.output, text)

    def lockout_demo(self):
        record = passwords.make_record("correct", 2000)
        guard = passwords.LoginGuard(record, 3, 30)
        attempts = [("aaa", 0), ("bbb", 1), ("ccc", 2), ("correct", 3), ("correct", 35)]
        lines = []
        for attempt, time_s in attempts:
            result = guard.try_login(attempt, now=time_s)
            lines.append("t=%3ds  essai '%s'  ->  %s" % (time_s, attempt, result))
        set_text(self.output, "\n".join(lines))

    def time_table(self):
        rate = passwords.hash_rate(50000)
        lines = ["Temps moyen pour trouver le mot de passe, SHA-256 simple à %d essais/s\n" % rate]
        for name, (charset, length) in CASES.items():
            space = passwords.search_space(charset, length)
            duration = passwords.human_time(passwords.estimate_seconds(space, rate))
            lines.append("%-34s %12d candidats  %s" % (name, space, duration))
        for length in (6, 8, 10, 12):
            space = len(passwords.FULL) ** length
            duration = passwords.human_time(passwords.estimate_seconds(space, rate))
            lines.append("%-34s %12.2e candidats  %s" % ("ensemble complet, %d car." % length,
                space, duration))
        set_text(self.output, "\n".join(lines))


def main():
    window = tk.Tk()
    window.title("Framework de chiffrement - SSAD TP1")
    window.geometry("820x720")
    tabs = ttk.Notebook(window)
    tabs.pack(fill="both", expand=True, padx=6, pady=6)
    tabs.add(MessagingTab(tabs), text="Messagerie")
    tabs.add(CipherAttackTab(tabs), text="Attaques sur les chiffrements")
    tabs.add(PasswordTab(tabs), text="Attaques sur mots de passe")
    tabs.add(ProtectionTab(tabs), text="Protection")
    window.mainloop()


if __name__ == "__main__":
    main()

```